🚀 TideX GitLab 企业内部协作全手册 (v1.0)

TideX 研发协同标准与工程质量规范

> 本手册旨在统一 TideX 内部研发协作标准，确保嵌入式、App 及后端项目的代码安全与高效流转。所有研发人员及测试人员入职后须严格遵守本规范。

📌 系统环境

- **访问域名**：http://TideX-gitlab.com
- **内网服务器 IP**：`192.168.1.212` (Ubuntu Server 32G)
- **网络要求**：必须连接 TideX 办公内网。若开启 VPN，须配置排除名单。

一、 环境初始化 (新员工必做)

1. 域名解析 (解决"网页无法访问")

- **Windows 用户**：右键以管理员身份运行公司提供的 `TideX_GitLab域名配置.bat`。
- **macOS / Linux 用户**：
  1. 终端执行：`sudo nano /etc/hosts`
  2. 末尾添加一行：`192.168.1.212 TideX-gitlab.com`
  3. 刷新 DNS：`sudo killall -HUP mDNSResponder`

2. VPN 冲突处理 (针对代理软件用户)

若使用 v2rayN、Clash 或公司 VPN，请在代理设置的"排除名单"中填入：`TideX-gitlab.com; 192.168.1.212` 并勾选 **"请勿将代理服务器用于本地地址"**，否则 Git 提交将报 403 或超时错误。

3. SSH 免密配置 (解决"频繁输入密码")

1. **生成密钥**：`ssh-keygen -t ed25519 -C "your_email@TideX.com"` (一路回车)。
2. **获取公钥**：打开 `~/.ssh/id_ed25519.pub`，复制全文。
3. **上传 GitLab**：点击个人头像 -> **Settings** -> **SSH Keys** -> **Add new key**。

二、 常用场景命令速查

1. 项目启动

目标场景	执行命令
首次克隆	`git clone git@TideX-gitlab.com:组名/项目名.git`

上传已有代码	`git init`
	`git remote add origin git@TideX-gitlab.com:组名/项目名.git`
	`git add . && git commit -m "init"`
	`git push -u origin main`

目标场景	执行命令
仓库搬家/改地址	`git remote set-url origin git@TideX-gitlab.com:组名/项目名.git`

2. 研发日常工作流

```
git pull origin main          # 每天上班第一件事：同步最新代码
git checkout -b feat-xxx      # 创建并切到功能分支
# ... 开发中 ...
git add .
git commit -m "[feat] 描述信息 (#Issue号)"
git push origin feat-xxx      # 推送到服务器，随后发起 Merge Request
```

3. 工作现场保存 (Stash)

正在开发功能 A，需紧急切到 main 修复 Bug：

```
git stash              # 暂存当前未提交的代码
git checkout main      # 切去修 Bug
git checkout feat-A    # 修复完切回功能分支
git stash pop          # 弹出刚才暂存的代码，继续开发
```

三、 软硬结合专项规范 (嵌入式 & App)

1. 代码卫生与 .gitignore

嵌入式 IDE (Keil, IAR等) 会产生大量中间文件。**禁止上传**这些冗余文件。

- **要求**：根目录必须配置 `.gitignore`，忽略 `*.obj`, `*.o`, `*.bin`, `*.hex`, `Listings/`, `Objects/` 等。
- **大文件管理**：固件、库文件或 App 资源超过 5MB 时，请联系管理员开启 **Git LFS**，严禁直接硬传。

2. 分支生命周期策略 (TideX Simplified GitFlow)

- **Main 分支**：受保护分支，对应稳定版本。**严禁直推代码。**
- **Feature 分支**：命名为 `feat/功能-姓名缩写`，任务完成后应及时删除。
- **Release 分支**：App 上线前从 main 切出，仅用于回归测试，不加新功能。

3. 联调协议同步

App 与 嵌入式端在联调前，必须先在 GitLab Wiki 或 `docs/` 下同步 **《通讯协议手册》**。涉及协议字段的 MR 需由双方 Leader 共同 Review。

四、 工业级协作与质量控制 (CR 规范)

1. 提交信息规范 (Commit Message)

统一格式：`[类型] 描述 (#Issue编号)`

- `[feat]`: 新功能 | `[fix]`: 修复 Bug | `[docs]`: 文档更新 | `[refactor]`: 重构。
- *示例：`[fix] 修复蓝牙自动断连问题 (#102)`*

2. 代码评审 (Code Review) 礼仪

- **MR 机制**：代码合并至 main 必须通过 Merge Request。
- **评审标记**：
  - `[Must]`：必须修改，否则不准合并。
  - `[Suggest]`：建议优化。
- **合并标准**：获得至少一名核心成员回复 **LGTM** (Looks Good To Me) 后，方可合并。

3. 版本标签 (Tagging)

- **原则**：凡是发布的版本（固件打包或 App 提审），必须在 main 分支打 Tag。
- **格式**：`v主版本.次版本.修订号_日期` (例：`v1.2.0_20231027`)。

五、 常见错误排查 (Troubleshooting)

现象	可能原因	解决方法
Could not resolve hostname	Hosts 映射失效	重新运行 `TideX_GitLab域名配置.bat`
Permission denied (publickey)	SSH 密钥不匹配	重新检查并上传本地 `.pub` 公钥至 GitLab
Rejected (non-fast-forward)	远程分支已有更新	执行 `git pull` 同步冲突后再尝试推送
网页显示 503	服务器启动中/过载	刚重启服务器后需等待 3-5 分钟后刷新
Push 提示：File too large	包含未授权的大文件	使用 Git LFS 或清理编译产生的临时文件

六、 岗位职责闭环管理

- **研发 (Developer)**：负责功能开发、分支清理及代码自测。
- **测试 (QA)**：负责在 Issues 提单。必须包含：**复现步骤、预期结果、关联固件/App版本号、截图**。验证通过后负责 Close Issue。
- **管理员 (Admin)**：负责账号权限审核、分支保护设置及每日定时备份。

📄 附录：TideX_GitLab 域名自动配置脚本 (Windows)

将以下内容保存为 `TideX_GitLab域名配置.bat`：

```
@echo off
set "ip=192.168.1.212"
set "domain=TideX-gitlab.com"
set "hostsfile=%SystemRoot%\system32\drivers\etc\hosts"

:: 检查管理员权限
```

```
net session >nul 2>&1
if %errorLevel% neq 0 (
    echo [错误] 请右键点击此脚本，选择“以管理员身份运行”！
    pause
    exit
)

echo 正在为 TideX GitLab 配置域名映射...
findstr /v "%domain%" "%hostsfile%" > "%temp%\hosts.tmp"
echo %ip%  %domain% >> "%temp%\hosts.tmp"
copy /y "%temp%\hosts.tmp" "%hostsfile%" >nul

echo.
echo ==========================================
echo 配置完成！
echo TideX GitLab 地址: http://%domain%
echo ==========================================
pause
```

结语：规范的建立是为了更自由的运转。请各位同事保持“代码整洁”意识，共同维护 TideX 的工程质量。